

Кросплатформне програмування

14

Створення GUI в Java

Бібліотеки для створення GUI

- **AWT (Abstract Window Toolkit)**

- **Swing**

- **JavaFX**

- Бібліотека JavaFX у порівнянні з AWT та Swing не потребує додаткової пам'яті або місця для зберігання
- В ній реалізовано прискорене графічне оброблення
- Бібліотека має додаткові віджети у порівнянні зі стандартним набором
- JavaFX має більшу кількість компонентів, ніж AWT, але меншу, ніж Swing

Ми розглянемо можливості бібліотеки Swing



Етапи створення GUI

- Створення графічних компонентів
- Налаштування параметрів компонентів
- Розміщення компонентів у контейнерах
- Оброблення подій



Створення компонентів

- Графічні компоненти можна створювати з використанням класів бібліотеки **Swing**
- Наприклад, для такого графічного компонента, як кнопка, можна використати клас **JButton**
- Щоб створити вікно використовують клас **JFrame**
- Об'єкт для текстової мітки може створюватись на основі класу **JLabel**

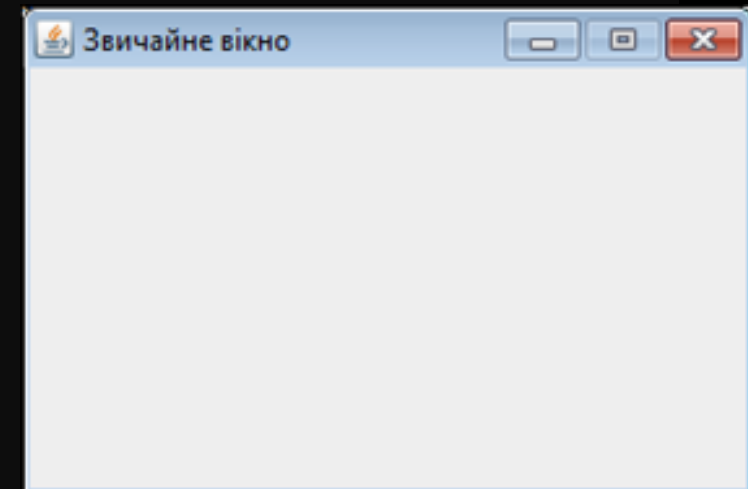


Приклад: пусте вікно

Код програми для відображення пустого вікна:

```
import javax.swing.JFrame;
class Demo{
    public static void main(String[] args){
        // Створення об'єкта вікна:
        JFrame wnd=new JFrame("Звичайне вікно");
        // Розміри вікна:
        wnd.setSize(300,200);
        // Позиція вікна на екрані:
        wnd.setLocation(250,250);
        // Вікно постійних розмірів:
        wnd.setResizable(false);
        // Реакція на натискання системної піктограми:
        wnd.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Відображення вікна на екрані:
        wnd.setVisible(true);
    }
}
```

Результат:



Принципи оброблення подій

- Існує програма з GUI, там є якісь графічні компоненти, і з цими компонентами може щось відбуватись (події)
- Події: натискання кнопки, наведення курсору миші на компонент, введення символу в поле вводу, вибір опції
- При виникненні події автоматично створюється об'єкт, який містить інформацію про цю подію
- Об'єкт події створюється на основі класу. В Java існує ієрархія класів для можливих подій, які можуть відбуватися з графічними компонентами
- Класи подій містять в своїй назві слово Event
- Для кожного компонента існує свій набір подій, на які компонент може в принципі реагувати (наприклад, кнопки можуть реагувати на події класу **ActionEvent**)
- Щоб компонент реагував на подію, необхідно в цьому компоненті зареєструвати обробник відповідної події (для різних подій - різні обробники)
- Обробник події - це об'єкт, створений на основі класу, який реалізує певний інтерфейс
- Якщо в компоненті зареєстровано обробник, то у цього обробника є певний набір методів, які й використовуються під час оброблення події



Корисні поради

- Щоб за іменем класу події визначити ім'я інтерфейсу (для реалізації в класі обробника), в назві класу події слід слово **Event** замінити на слово **Listener**
- Наприклад, якщо йдеться про подію класу **ActionEvent**, то відповідний інтерфейс називається **ActionListener**
- Щоб дізнатись ім'я методу, яким в компоненті реєструється обробник, в назві інтерфейсу (який реалізується в класі обробника) на початку слід розмістити слово **add**
- Наприклад, обробники подій класу **ActionEvent** створюються на основі класів, які реалізують інтерфейс **ActionListener**, а реєструються в компоненті за допомогою методу **addActionListener()**



Пояснення

- В інтерфейсі **ActionListener** оголошений всього один абстрактний метод **actionPerformed()** (тобто інтерфейс є функціональним)
- Метод **actionPerformed()** не повертає результат. У метода один аргумент - посилання на об'єкт класу **ActionEvent** (об'єкт події)
- Якщо в компоненті реєструється обробник події класу **ActionEvent**, то обробником має бути об'єкт класу, який реалізує інтерфейс **ActionListener**
- Реєструється обробник методом **addActionListener()**
- При виникненні (з компонентом) події вказаного типу з об'єкта-обробника викликається метод **actionPerformed()**, а аргументом методу передається посилання на об'єкт події

Приклад: обробник на основі анонімного класу

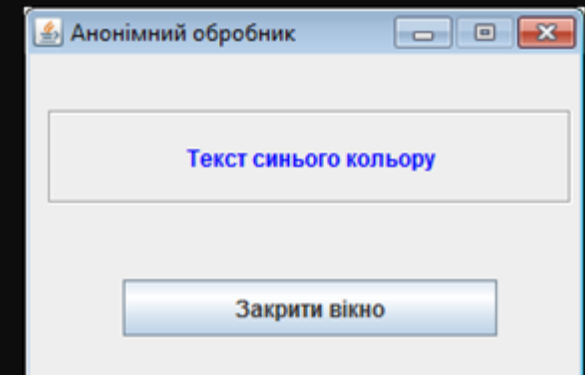
Код програми:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{
    MyFrame(String name){
        super(name);
        setBounds(250,250,300,200);
        setResizable(false);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(null);
        JLabel lbl=new JLabel(
            "Текст синього кольору",
            JLabel.CENTER
        );
        lbl.setBounds(10,30,280,50);
        lbl.setForeground(Color.BLUE);
        lbl.setBorder(BorderFactory.createEtchedBorder());
        add(lbl);
        JButton btn=new JButton("Закрити вікно");
        btn.setBounds(50,120,200,30);
        btn.setFocusPainted(false);
        btn.addActionListener(new ActionListener(){
            public void actionPerformed(ActionEvent e){
                System.exit(0);
            }
        });
        add(btn);
        setVisible(true);
    }
}

class Demo{
    public static void main(String[] args){
        new MyFrame("Анонімний обробник");
    }
}
```

Результат:



Приклад: обробник на основі лямбда- виразу

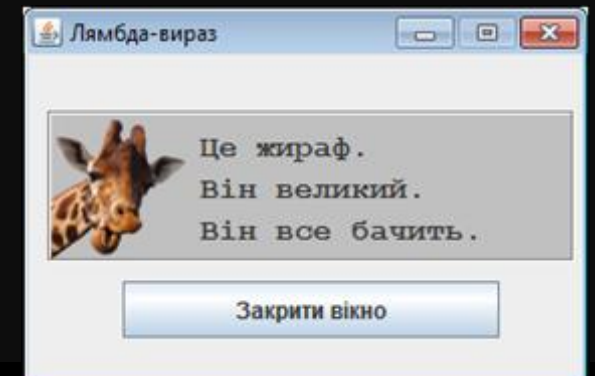
Код програми:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame{    // Клас для створення вікна
    MyFrame(String name){        // Конструктор
        super(name);            // Виклик конструктора суперкласу
        setBounds(250,250,300,200); // Позиція і розміри вікна
        setResizable(false);      // Вікно постійних розмірів
        setDefaultCloseOperation(EXIT_ON_CLOSE); // Реакція на натискання системної піктограми
        setLayout(null); // Відключення менеджера компоновки
        ImageIcon img=new ImageIcon("d:/books/pictures/giraffe.png"); // Об'єкт зображення (для відображення в мітці)
        String txt="<html>Це жираф.<br>Він великий.<br>Він все бачить.</html>"; // Текстове значення для відображення в мітці
        JLabel lbl=new JLabel(txt,img,JLabel.LEFT); // Створення об'єкта мітки
        Font F=new Font(Font.MONOSPACED,Font.BOLD,16); // Створення об'єкта шрифту
        lbl.setFont(F); // Застосування шрифту до мітки
        lbl.setBounds(10,30,280,80); // Позиція і розміри мітки
        lbl.setOpaque(true); // Перехід в режим непрозорості мітки
        lbl.setBackground(Color.LIGHT_GRAY); // Світло-сірий колір для фону мітки
        lbl.setBorder(BorderFactory.createEtchedBorder()); // Рамка навколо мітки
        add(lbl); // Додавання мітки у вікно
        JButton btn=new JButton("Закрити вікно"); // Додавання мітки у вікно
        btn.setBounds(50,120,200,30); // Позиція і розміри кнопки
        btn.setFocusPainted(false); // Відмінюється режим відображення рамки фокусу
        btn.addActionListener(e->{System.exit(0);}); // Реєстрація в кнопці обробника на основі лямбда-виразу
        add(btn); // Додавання кнопки у вікно
        setVisible(true); // Відображення вікна на екрані
    }
}

class Demo{
    public static void main(String[] args){
        // Створення об'єкта вікна:
        new MyFrame("Лямбда-вираз");
    }
}
```

Результат:



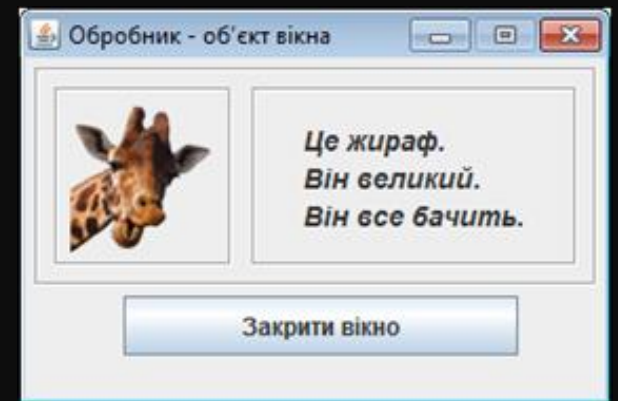
```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

class MyFrame extends JFrame implements ActionListener{
    public void actionPerformed(ActionEvent e){
        System.exit(0);
    }
    MyFrame(String name){
        super(name);
        setBounds(250,250,300,200);
        setResizable(false);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(null);
        JPanel pnl=new JPanel(); // Створення об'єкта панелі
        pnl.setBounds(5,5,285,110);
        pnl.setBorder(BorderFactory.createEtchedBorder());
        pnl.setLayout(null);
        ImageIcon img=new ImageIcon("d:/books/pictures/giraffe.png");
        String txt="<html>Це жираф.<br>Він великий.<br>Він все бачить.</html>";
        JLabel imgLbl=new JLabel(img); // Створення об'єкта мітки з зображенням
        imgLbl.setBounds(10,10,90,90);
        imgLbl.setBorder(BorderFactory.createEtchedBorder());
        JLabel txtLbl=new JLabel(txt,JLabel.CENTER); // Створення об'єкта для мітки з текстом
        txtLbl.setBounds(110,10,165,90);
        txtLbl.setBorder(BorderFactory.createEtchedBorder());
        JButton btn=new JButton("Закрити вікно");
        btn.setBounds(50,120,200,30);
        btn.setFocusPainted(false);
        btn.addActionListener(this); // Реєстрація в кнопці обробником об'єкта вікна
        Font F=new Font(btn.getFont().getName(),Font.BOLD|Font.ITALIC,btn.getFont().getSize()+2);
        txtLbl.setFont(F);
        pnl.add(imgLbl);
        pnl.add(txtLbl);
        add(pnl);
        add(btn);
        setVisible(true);
    }
}

class Demo{
    public static void main(String[] args){
        new MyFrame("Обробник - об'єкт вікна");
    }
}
```

Приклад: обробник на основі об'єкта вікна

Результат:





Пакети бібліотеки Swing

Пакет	Описання
<code>javax.swing</code>	Пакет містить утиліти для реалізації "легких" компонентів інтерфейсу
<code>javax.swing.border</code>	Пакет містить класи й інтерфейси для відображення спеціальних рамок навколо "легких" компонентів
<code>javax.swing.colorchooser</code>	Пакет містить класи й інтерфейси для реалізації утиліти вибору кольору в режимі діалогового окна з кольоровою палітрою
<code>javax.swing.event</code>	Пакет з утилітами для оброблення подій для "легких" компонентів
<code>javax.swing.filechooser</code>	Пакет містить класи й інтерфейси для реалізації системи вибору файлів у режимі діалогового вікна
<code>javax.swing.plaf</code>	Пакет містить утиліти (інтерфейс та абстрактні класи) для підтримки системи загальних стилів що підключаються (pluggable look-and-feel, що в скороченому вигляді дає plaf)
<code>javax.swing.plaf.basic</code>	Пакет для підтримання базового стилю
<code>javax.swing.plaf.metal</code>	Пакет для підтримання використовуваного в Java за замовчуванням стилю (називається Metal)
<code>javax.swing.plaf.multi</code>	Пакет для підтримки декількох комбінованих стилів
<code>javax.swing.plaf.nimbus</code>	Пакет для підтримки міжплатформеного стилю Nimbus
<code>javax.swing.plaf.synth</code>	Пакет для підтримки стилю Synth
<code>javax.swing.table</code>	Пакет містить класи й об'єкти для роботи з таблицями
<code>javax.swing.text</code>	Пакет з класами та інтерфейсами для роботи з текстовими компонентами
<code>javax.swing.text.html</code>	Пакет містить утиліти для роботи з текстовими HTML-редакторами
<code>javax.swing.text.html.parser</code>	Пакет для підтримки використовуваного за замовчуванням HTML-аналізатора
<code>javax.swing.text.rtf</code>	Пакет містить клас (називається RTFEditorKit) для створення текстового rtf-редактора
<code>javax.swing.tree</code>	Пакет містить класи та інтерфейси для роботи з деревами
<code>javax.swing.undo</code>	Пакет з утилітами для підтримки системи відміни й повторення дій для застосунків типу текстових редакторів



Класи пакета javax.swing

AbstractAction	JCheckBox	JSpinner.DateEditor	ScrollPaneLayout
AbstractButton	JCheckBoxMenuItem	JSpinner.DefaultEditor	ScrollPaneLayout.UIResource
AbstractCellEditor	JColorChooser	JSpinner.ListEditor	SizeRequirements
AbstractListModel	JComboBox	JSpinner.NumberEditor	SizeSequence
AbstractSpinnerModel	JComponent	JSplitPane	SortingFocusTraversalPolicy
ActionMap	JDesktopPane	JTabbedPane	SpinnerDateModel
BorderFactory	JDialog	JTable	SpinnerListModel
Box	JEditorPane	JTable.DropLocation	SpinnerNumberModel
Box.Filler	JFileChooser	JTextArea	Spring
BoxLayout	JFormattedTextField	JTextField	SpringLayout
ButtonGroup	JFormattedTextField.AbstractFormatter	JTextPane	SpringLayout.Constraints
CellRendererPane	JFormattedTextField.AbstractFormatterFactory	JToggleButton	SwingUtilities
ComponentInputMap	JFrame	JToggleButton.ToggleButtonModel	SwingWorker
DebugGraphics	JInternalFrame	JToolBar	Timer
DefaultBoundedRangeModel	JInternalFrame.JDesktopIcon	JToolBar.Separator	ToolTipManager
DefaultButtonModel	JLabel	JToolTip	TransferHandler
DefaultCellEditor	JLayer	JTree	TransferHandler.DropLocation
DefaultComboBoxModel	JLayeredPane	JTree.DropLocation	TransferHandler.TransferSupport
DefaultDesktopManager	JList	JTree.DynamicUtilTreeNode	UIDefaults
DefaultFocusManager	JList.DropLocation	JTree.EmptySelectionModel	UIDefaults.LazyInputMap
DefaultListCellRenderer	JMenu	JViewport	UIDefaults.ProxyLazyValue
DefaultListCellRenderer.UIResource	JMenuBar	JWindow	UIManager
DefaultListModel	JMenuItem	KeyStroke	UIManager.LookAndFeelInfo
DefaultListSelectionModel	JOptionPane	LayoutFocusTraversalPolicy	ViewportLayout
DefaultRowSorter	JPanel	LayoutStyle	
DefaultRowSorter.ModelWrapper	JPasswordField	LookAndFeel	
DefaultSingleSelectionModel	JPopupMenu	MenuSelectionManager	
FocusManager	JPopupMenu.Separator	OverlayLayout	
GrayFilter	JProgressBar	Popup	
GroupLayout	JRadioButton	PopupFactory	
ImageIcon	JRadioButtonMenuItem	ProgressMonitor	
InputMap	JRootPane	ProgressMonitorInputStream	
InputVerifier	JScrollBar	RepaintManager	
InternalFrameFocusTraversalPolicy	JScrollPane	RowFilter	
JApplet	JSeparator	RowFilter.Entry	
JButton	JSlider	RowSorter	
	JSpinner	RowSorter.SortKey	

Класи Swing для реалізації компонентів GUI

Клас	Описання
AbstractButton	Суперклас для підкласів, через які реалізуються кнопки та елементи меню
BorderFactory	Клас, за допомогою якого задається тип границі для елемента інтерфейсу
BoxLayout	Клас для створення менеджера компоновки
ButtonGroup	Клас для створення групи кнопок
ImageIcon	Клас для створення піктограм (на основі зображень)
JApplet	Клас для створення аплетів
JButton	Клас для створення кнопок
JCheckBox	Клас для створення опцій
JCheckBoxMenuItem	Клас для створення опційного елемента меню
JColorChooser	Клас для роботи з діалоговим вікном вибору кольору

Класи Swing для реалізації компонентів GUI

Клас	Описання
JComboBox	Клас для реалізації динамічного списку
JComponent	Суперклас для великої кількості класів бібліотеки Swing, через які реалізуються елементи графічного інтерфейсу
JDesktopPane	Клас використовується при створенні багатодокументних застосунків
JDialog	Клас для створення діалогових вікон
JEditorPane	Клас для створення текстової області, яка дозволяє відображати та редагувати текст у форматі HTML та RTF
JFileChooser	Клас для роботи з діалоговим вікном вибору файлу
JFormattedTextField	Клас для реалізації текстового поля введення з підтримкою форматування

Класи Swing для реалізації компонентів GUI

Клас	Описання
JFrame	Клас для реалізації вікна форми
JLabel	Клас для створення мітки
JList	Клас для створення такого елемента інтерфейсу, як список вибору
JMenu	Клас для створення пунктів головного меню (або підпунктів меню)
JMenuBar	Клас для створення панелі меню
JMenuItem	Клас для створення елементів (команд) меню
JOptionPane	Клас з утилітами для використання стандартних діалогових вікон
JPanel	Клас для реалізації панелі
JPasswordField	Клас для створення поля введення пароля
JPopupMenu	Клас для створення контекстного меню

Класи Swing для реалізації компонентів GUI

Клас	Описання
JProgressBar	Клас для створення такого елемента, як індикатор процесу (індикатор степені завершення процесу)
JRadioButton	Клас для реалізації перемикача
JRadioButtonMenuItem	Клас для реалізації перемикача-елемента меню
JRootPane	Клас корневої панелі для віконних елементів інтерфейсу
JScrollBar	Клас для створення полоси прокрутки
JScrollPane	Клас для створення панелі з полоєю прокрутки
JSeparator	Клас для створення роздільника (декоративний елемент, що використовується в меню)
JSlider	Клас для створення слайдера (лінійного регулятора)

Класи Swing для реалізації компонентів GUI

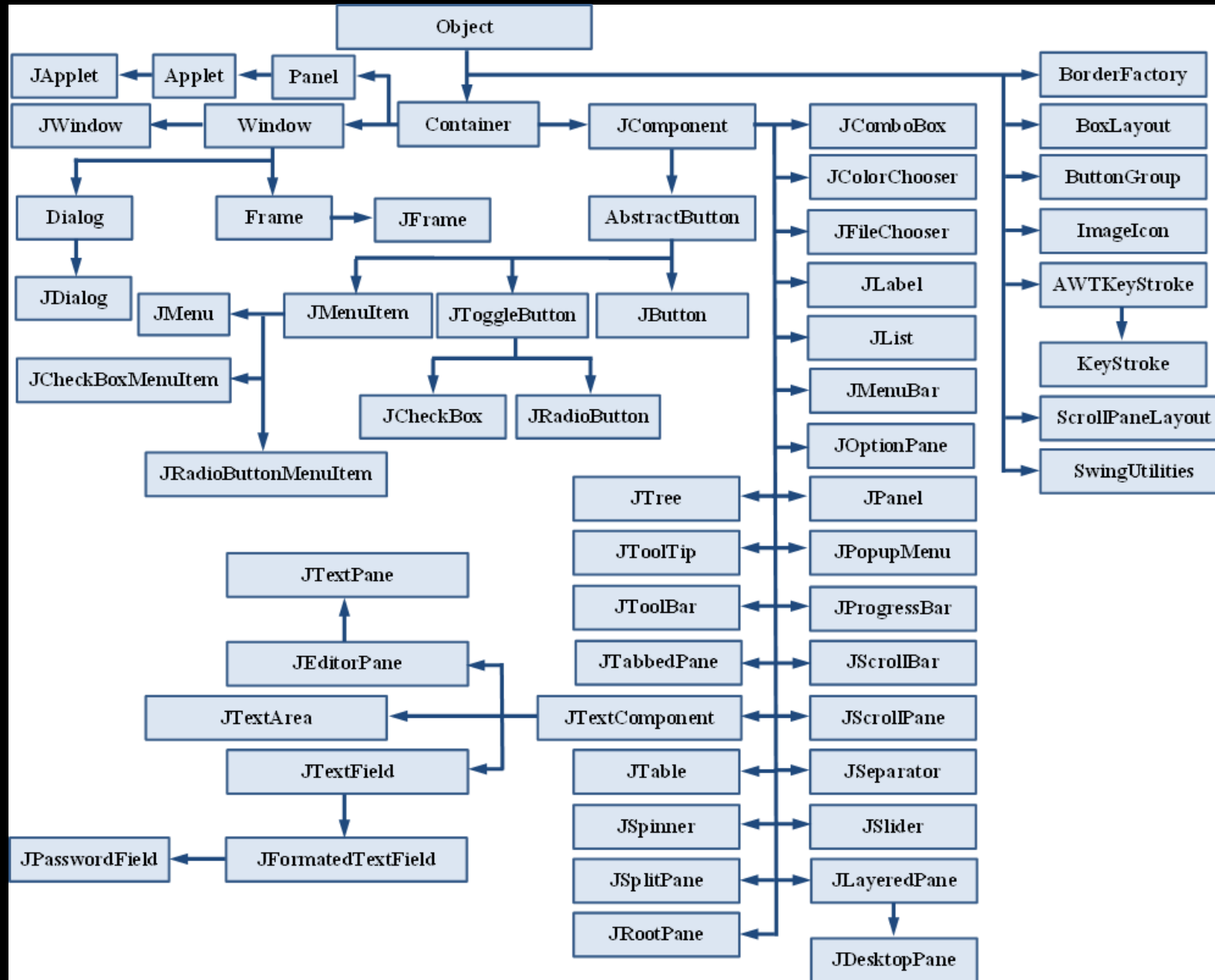
Клас	Описання
JSpinner	Клас для створення спінера (поле зі значенням і піктограмами для збільшення/зменшення цього значення)
JSplitPane	Клас для створення панелі з роздільником на дві частини
JTabbedPane	Клас для творення панелі з вкладками
JTable	Клас для реалізації таблиць
JTextArea	Клас для створення текстової області, що дозволяє вводити й відображати декілька рядків тексту
TextField	Клас для створення текстового поля вводу
JTextPane	Клас для створення текстової області з розширеними можливостями редагування й форматування

Класи Swing для реалізації компонентів GUI

Клас	Описання
JToggleButton	Клас для створення опційної кнопки
JToolBar	Клас для створення панелі інструментів
JToolTip	Клас для створення контекстної підказки для елементів інтерфейсу
JTree	Клас для роботи з ієрархічними "деревами"
JWindow	Клас-контейнер "простого" вікна (без назви й системних компонентів)
KeyStroke	Клас призначено для використання при обробці дій з клавіатурою
ScrollPaneLayout	Клас для реалізації менеджера компоновки для панелі з прокруткою
SwingUtilities	Клас містить набір утиліт для роботи з елементами, що реалізуються засобами бібліотеки Swing

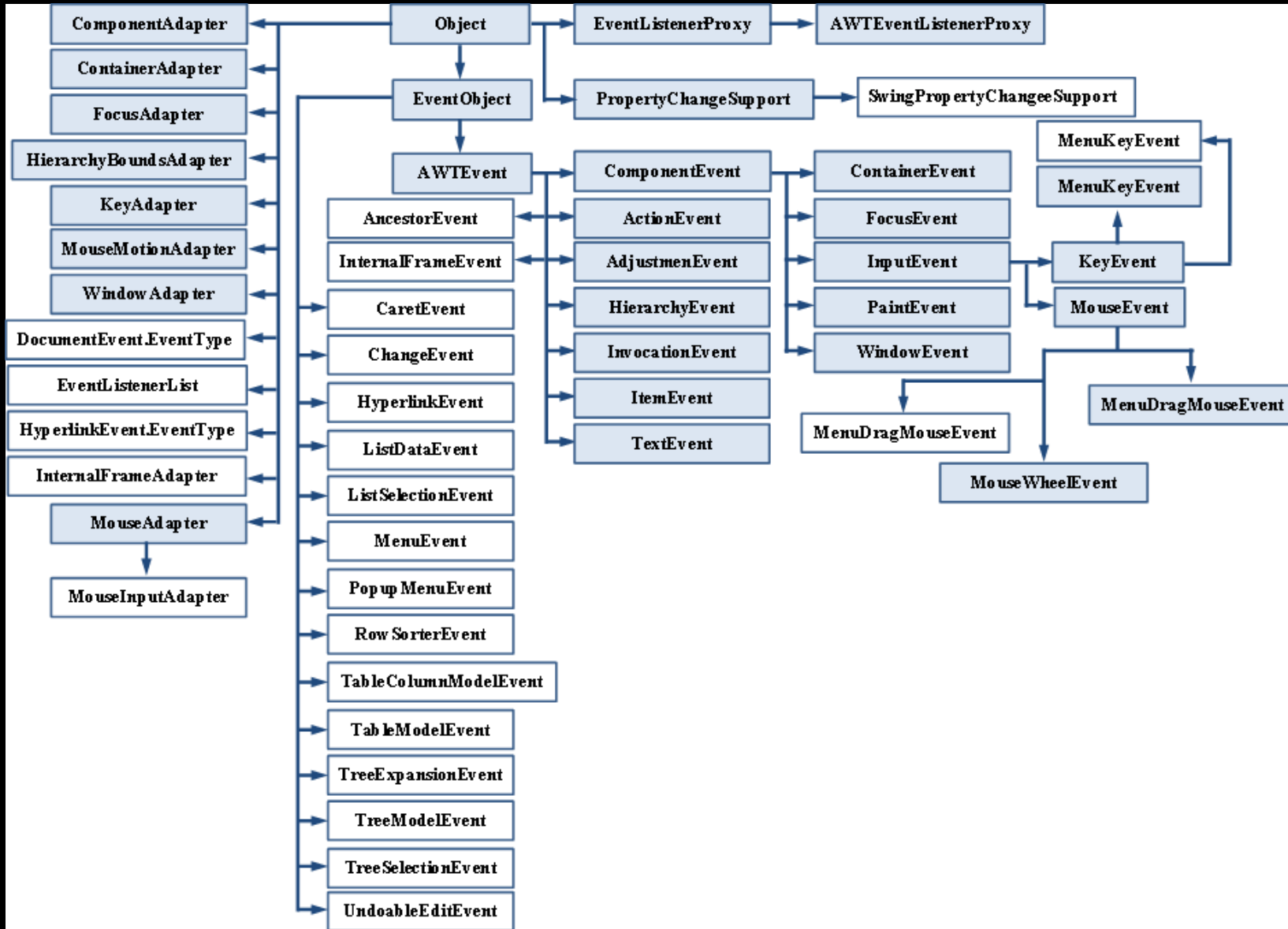


Ієрархія класів Swing





Ієрархія класів подій



Світлий колір - пакет
javax.swing.event

Темний колір - пакет
java.awt.event



Класи пакету java.awt.event

ActionEvent

AdjustmentEvent

AWTEventListenerProxy

ComponentAdapter

ComponentEvent

ContainerAdapter

ContainerEvent

FocusAdapter

FocusEvent

HierarchyBoundsAdapter

HierarchyEvent

InputEvent

InputMethodEvent

InvocationEvent

ItemEvent

KeyAdapter

KeyEvent

MouseAdapter

MouseEvent

MouseMotionAdapter

MouseWheelEvent

PaintEvent

TextEvent

WindowAdapter

WindowEvent

Класи пакету java.swing.event

AncestorEvent

CaretEvent

ChangeEvent

DocumentEvent.EventType

EventListenerList

HyperlinkEvent

HyperlinkEvent.EventType

InternalFrameAdapter

InternalFrameEvent

ListDataEvent

ListSelectionEvent

MenuDragMouseEvent

MenuEvent

MenuKeyEvent

MouseInputAdapter

PopupMenuEvent

RowSorterEvent

SwingPropertyChangeSupport

TableColumnModelEvent

TableModelEvent

TreeExpansionEvent

TreeModelEvent

TreeSelectionEvent

UndoableEditEvent



Деякі класи подій

Клас	Описання
ActionEvent	Подія - індикатор того, що користувач виконав з елементом інтерфейсу характерну (для цього елемента) дію
ComponentAdapter	Абстрактний клас-адаптер для створення обробника для подій компонентів інтерфейсу
ComponentEvent	Подія, пов'язана зі зміною характеристик (властивостей) компонента
FocusAdapter	Абстрактний клас-адаптер для оброблення події, пов'язаної з втратою або отриманням фокусу
FocusEvent	Подія, пов'язана з втратою або отриманням фокусу компонентом
InputEvent	Ключовий клас для класів подій, пов'язаних з введенням
ItemEvent	Подія, пов'язана зі зміною стану елемента
KeyAdapter	Абстрактний клас-адаптер для оброблення подій, пов'язаних з клавіатурою (з натисканням клавіш)
KeyEvent	Подія для компонента, пов'язана з натисканням клавіші
MouseAdapter	Абстрактний клас-адаптер, призначений для оброблення подій, пов'язаних з діями мишкою
MouseEvent	Подія, пов'язана з діями мишкою
MouseMotionAdapter	Абстрактний клас-адаптер для оброблення подій, пов'язаних з переміщенням мишки

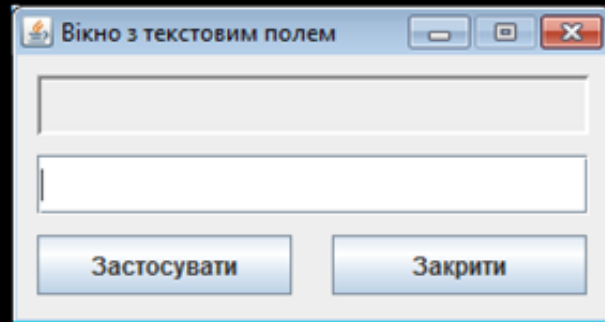


Деякі класи подій

Клас	Описання
MouseEvent	Подія, пов'язана з обертанням колеса мишки в області компонента інтерфейсу
PaintEvent	Подія, пов'язана з малюванням
TextEvent	Подія, пов'язана зі зміною тексту компоненту
WindowAdapter	Абстрактний клас-адаптер для оброблення подій вікна
WindowEvent	Подія, пов'язана зі зміною статусу вікна
CaretEvent	Подія, пов'язана зі зміною положення текстової каретки
ChangeEvent	Подія, пов'язана зі зміною стану компонента
ListDataEvent	Подія, пов'язана зі зміною вмісту списку
ListSelectionEvent	Подія, пов'язана зі зміною вибору користувача (в компонентах типу списку вибору)
MenuEvent	Подія, пов'язана з дією вибору/відміни меню
PopupMenuEvent	Подія, пов'язана з контекстним меню

Деякі приклади

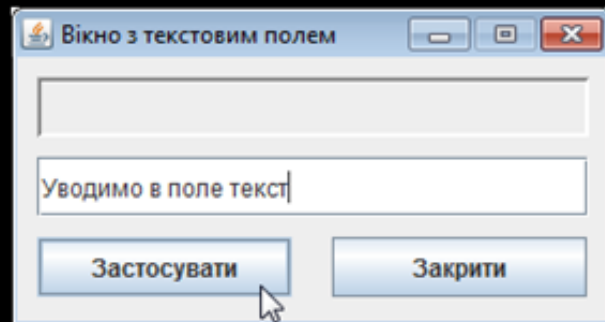
Результат: зчитування значення поля



Вікно з текстовим полем

Two empty text input fields.

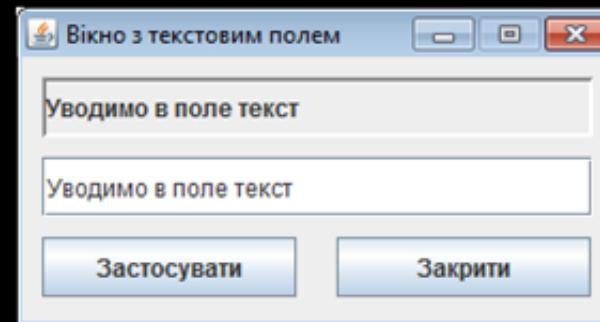
Застосувати Закрити



Вікно з текстовим полем

Уводимо в поле текст

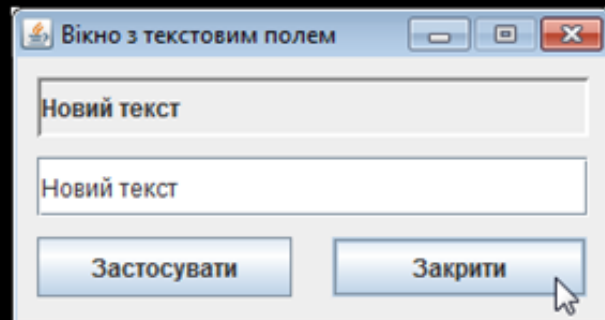
Застосувати Закрити



Вікно з текстовим полем

Уводимо в поле текст

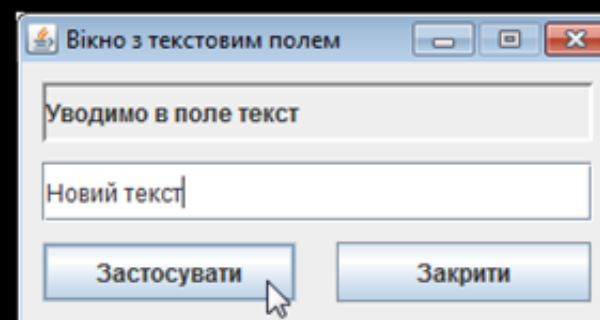
Застосувати Закрити



Вікно з текстовим полем

Новий текст

Застосувати Закрити



Вікно з текстовим полем

Новий текст

Застосувати Закрити

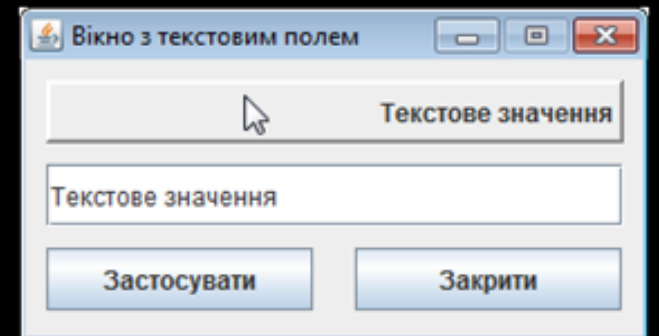
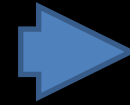
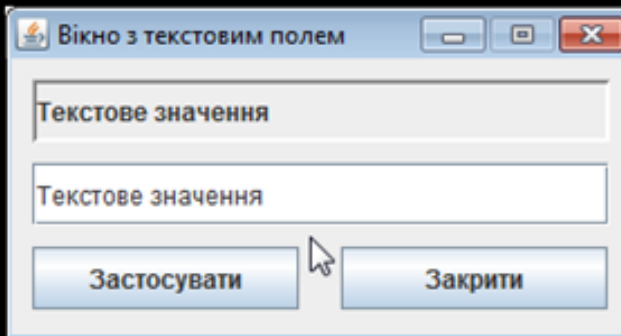
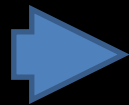
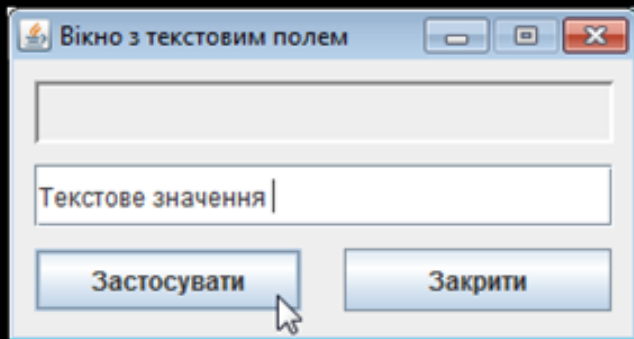
Приклад: зчитування значення поля

```
// Імпорт класів:
import javax.swing.*;
import javax.swing.border.*;
import java.awt.event.*;
// Клас для створення вікна:
class MyFrame extends JFrame{
    // Мітка:
    private JLabel L;
    // Текстове поле:
    private JTextField T;
    // Конструктор:
    MyFrame() {
        // Виклик конструктора суперкласу:
        super("Вікно з текстовим полем");
        // Значення для розмірів вікна:
        int w=300,h=160;
        // Позиція і розміри вікна:
        setBounds(250,250,w,h);
        // Реакція на натискання системної піктограми:
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        // Вікно постійних розмірів:
        setResizable(false);
        // Відключення менеджера компоновки:
        setLayout(null);
        // Створення мітки:
        L=new JLabel();
        // Позиція і розміри мітки:
        L.setBounds(10,10,w-25,30);
        // Виділення мітки за допомогою ефекту "втискування":
        L.setBorder(
            BorderFactory.createBevelBorder(
                BevelBorder.LOWERED
            )
        );
        // Створення текстового поля:
        T=new JTextField();
        // Позиція і розміри поля:
        T.setBounds(L.getX(),50,L.getWidth(),30);
        // Ширина кнопок:
        int bw=(T.getWidth()-20)/2;
        // Створення першої кнопки:
        JButton appB=new JButton("Застосувати");
```

```
// Позиція і розміри першої кнопки:
appB.setBounds(T.getX(),90,bw,30);
// Відміна режиму відображення рамки фокусу
// для першої кнопки:
appB.setFocusPainted(false);
// Обробник події для першої кнопки:
appB.addActionListener(e->L.setText(T.getText()));
// Створення другої кнопки:
JButton extB=new JButton("Закрити");
// Позиція і розміри другої кнопки:
extB.setBounds(
    appB.getX()+appB.getWidth()+20,
    appB.getY(),
    appB.getWidth(),
    appB.getHeight()
);
// Відміна режиму відображення рамки фокусу
// для другої кнопки:
extB.setFocusPainted(false);
// Обробник події для другої кнопки:
extB.addActionListener(e->System.exit(0));
// Додавання мітки у вікно:
add(L);
// Додавання поля у вікно:
add(T);
// Додавання першої кнопки у вікно:
add(appB);
// Додавання другої кнопки у вікно:
add(extB);
// Відображення вікна:
setVisible(true);
}
}
// Головний клас:
class TextFieldDemo{
    // Головний метод:
    public static void main(String[] args){
        // Створення вікна:
        new MyFrame();
    }
}
```

Приклад: спільний обробник

Мітка реагує на події: у випадку наведення курсору миші на область мітки вона відображається з застосуванням ефекту "підняття", а текст в мітці (допоки над областю мітки знаходиться курсор) вирівнюється за правим краєм. Якщо курсор миші прибрати з області мітки, то мітка відображатиметься з використанням ефекту "втискування", текст вирівнюватиметься за лівим краєм. Тут ми маємо справу з подією класу **MouseEvent**, пов'язаною з діями з мишкою





Приклад: спільний обробник

```
// Імпорт класів:
import javax.swing.*;
import javax.swing.border.*;
import java.awt.event.*;
// Клас для створення вікна реалізує
// інтерфейси ActionListener та MouseListener:
class MyFrame extends JFrame implements
ActionListener,MouseListener{
    // Мітка:
    private JLabel L;
    // Текстове поле:
    private JTextField T;
    // Назви кнопок:
    private String apply="Застосувати";
    private String exit="Закрити";
    // Метод для обробки натискання кнопки:
    public void actionPerformed(ActionEvent e){
        // Визначення назви кнопки, на якій
        // відбулась подія:
        String txt=e.getActionCommand();
        // Якщо перша кнопка:
        if(txt.equals(apply)){
            // Присвоєння мітці тексту з поля:
            L.setText(T.getText());
        } // Якщо друга кнопка:
        else{
            // Завершення виконання програми:
            System.exit(0);
        }
    }
    // Метод виконується, коли курсор миші покидає
    // область компоненту:
    public void mouseExited(MouseEvent e){
        // Застосування до мітки ефекту "втискування":
        L.setBorder(
            BorderFactory.createBevelBorder(
                BevelBorder.LOWERED)
        );
        // Застосування вирівнювання за лівим краєм
        // для тексту в мітці:
        L.setHorizontalAlignment(JLabel.LEFT);
    }
}
```

```
// Метод викликається, коли курсор опиняється над
// областю компоненту:
public void mouseEntered(MouseEvent e){
    // Для мітки застосовується ефект "підняття":
    L.setBorder(
        BorderFactory.createBevelBorder(
            BevelBorder.RAISED)
    );
    // Застосування вирівнювання за правим краєм
    // для тексту в мітці:
    L.setHorizontalAlignment(JLabel.RIGHT);
}
// Методи з інтерфейсу MouseListener
// з пустою реалізацією:
public void mouseReleased(MouseEvent e){}
public void mousePressed(MouseEvent e){}
public void mouseClicked(MouseEvent e){}
// Конструктор:
MyFrame(){
    // Виклик конструктора суперкласу:
    super("Вікно з текстовим полем");
    // Значення для розмірів вікна:
    int w=300,h=160;
    // Позиція і розміри вікна:
    setBounds(250,250,w,h);
    // Реакція на натискання системної піктограми:
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    // Вікно постійних розмірів:
    setResizable(false);
    // Відключення менеджера компоновки:
    setLayout(null);
    // Створення мітки:
    L=new JLabel();
    // Позиція і розміри мітки:
    L.setBounds(10,10,w-25,30);
    // Виділення мітки за допомогою ефекту "втискування":
    L.setBorder(
        BorderFactory.createBevelBorder(
            BevelBorder.LOWERED)
    );
}
```

Приклад: спільний обробник (продовження)

Код програми:

```
// Реєстрація обробника у мітці:
L.addMouseListener(this);
// Створення текстового поля:
T=new JTextField();
// Позиція і розміри поля:
T.setBounds(L.getX(),50,L.getWidth(),30);
// Ширина кнопок:
int bw=(T.getWidth()-20)/2;
// Створення першої кнопки:
JButton appB=new JButton(apply);
// Позиція і розміри першої кнопки:
appB.setBounds(T.getX(),90,bw,30);
// Відміна режиму відображення рамки фокусу
// для першої кнопки:
appB.setFocusPainted(false);
// Реєстрація об'єкта вікна обробником
// події для першої кнопки:
appB.addActionListener(this);
// Створення другої кнопки:
JButton extB=new JButton(exit);
// Позиція і розміри другої кнопки:
extB.setBounds(
    appB.getX()+appB.getWidth()+20,
    appB.getY(),
    appB.getWidth(),
    appB.getHeight()
);
```

```
// Відміна режиму відображення рамки фокусу
// для другої кнопки:
extB.setFocusPainted(false);
// Реєстрація об'єкта вікна обробником
// події для другої кнопки:
extB.addActionListener(this);
// Додавання мітки у вікно:
add(L);
// Додавання поля у вікно:
add(T);
// Додавання першої кнопки у вікно:
add(appB);
// Додавання другої кнопки у вікно:
add(extB);
// Відображення вікна:
setVisible(true);
}
}
// Головний клас:
class MoreTextFieldDemo{
    // Головний метод:
    public static void main(String[] args){
        // Створення вікна:
        new MyFrame();
    }
}
```

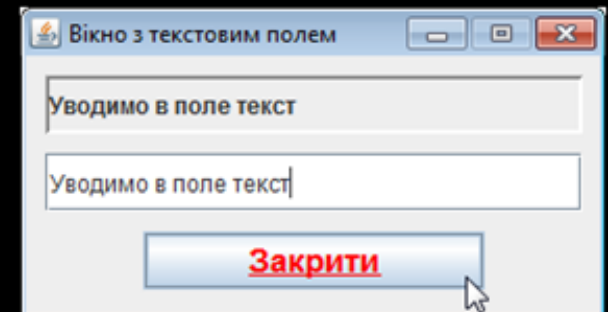
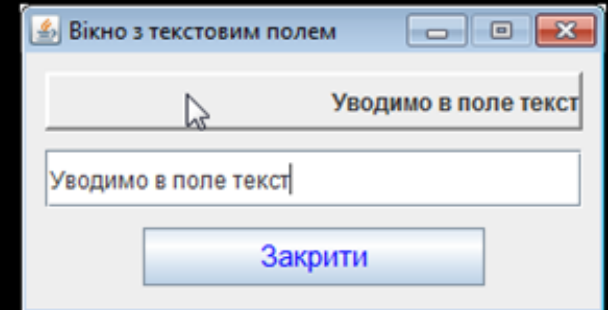
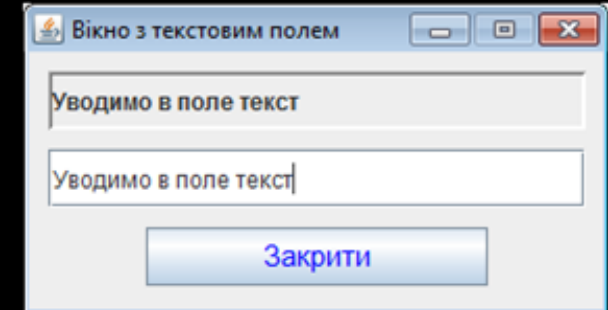
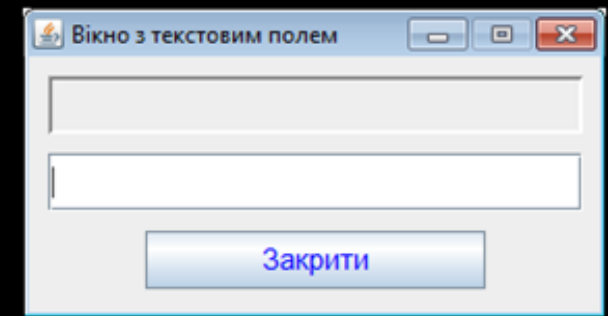


Приклад: обробник для текстового поля

Під час уведення тексту в поле текст автоматично відображається у мітці. Кнопка одна, і якщо її натиснути, то вікно закриється, а програма завершить виконання. Назва кнопки відображається синім кольором, а у випадку наведення курсору миші на кнопку назва кнопки відображається жирним шрифтом збільшеного розміру, текст червоного кольору і використовується ефект підкреслення. Якщо курсор миші прибрати з області кнопки, то кнопка повертається у свій вихідний стан. Також у випадку наведення курсору миші на мітку вона відображається з ефектом "підняття" і текст в мітці вирівнюється за правим краєм

Код програми:

```
// Імпорт класів:
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
// Клас для створення вікна реалізує інтерфейси
// ActionListener, MouseListener та KeyListener:
class MyFrame extends JFrame implements ActionListener, MouseListener, KeyListener{
    // Мітка:
    private JLabel L;
    // Текстове поле:
    private JTextField T;
    // Кнопка:
    private JButton B;
    // Тип шрифту для кнопки:
    private String name="Arial";
    // Розмір шрифту для кнопки:
    private int size=15;
    // Назва кнопки:
    private String exit="Закрити";
    // Метод для обробки натискання кнопки:
    public void actionPerformed(ActionEvent e){
        // Завершення виконання програми:
        System.exit(0);
    }
}
```



```
// Метод викликається, коли курсор миші покидає область компоненту:
public void mouseExited(MouseEvent e){
    // Якщо подія відбулась на мітці:
    if(e.getSource()==L){
        // Застосування до мітки ефекту "втискування":
        L.setBorder(
            BorderFactory.createBevelBorder(
                BevelBorder.LOWERED)
        );
        // Застосування вирівнювання за лівим краєм
        // для тексту мітки:
        L.setHorizontalAlignment(JLabel.LEFT);
    } // Якщо подія відбулась на кнопці:
    else{
        // Текст (звичайний) для кнопки:
        B.setText(exit);
        // Синій колір для тексту кнопки:
        B.setForeground(Color.BLUE);
        // Шрифт (звичайний) для кнопки:
        B.setFont(new Font(name,Font.PLAIN,size));
    }
}
// Метод викликається, коли курсор опиняється над областю компоненту:
public void mouseEntered(MouseEvent e){
    // Якщо подія відбулась на мітці:
    if(e.getSource()==L){
        // Для мітки застосовується ефект "підняття":
        L.setBorder(
            BorderFactory.createBevelBorder(
                BevelBorder.RAISED)
        );
        // Застосування вирівнювання за правим краєм
        // для тексту в мітці:
        L.setHorizontalAlignment(JLabel.RIGHT);
    } // Якщо подія відбулась на кнопці:
    else{
        // Текст (підкреслений) для кнопки:
        B.setText("<html><u>"+exit+"</u></html>");
        // Червоний колір для тексту кнопки:
        B.setForeground(Color.RED);
        // Шрифт (жирний) для тексту кнопки:
        B.setFont(new Font(name,Font.BOLD,size+2));
    }
}
```

Приклад: обробник для текстового поля

```
// Методи з інтерфейсу MouseListener
// з пустою реалізацією:
public void mouseReleased(MouseEvent e){}
public void mousePressed(MouseEvent e){}
public void mouseClicked(MouseEvent e){}
// Метод викликається, коли відпускається клавіша
// на клавіатурі:
public void keyReleased(KeyEvent e){
    // До мітки застосовується текст з текстового поля:
    L.setText(T.getText());
}
// Методи з інтерфейсу KeyListener
// з пустою реалізацією:
public void keyPressed(KeyEvent e){}
public void keyTyped(KeyEvent e){}
// Конструктор:
MyFrame(){
    // Виклик конструктора суперкласу:
    super("Вікно з текстовим полем");
    // Позиція і розміри вікна:
    setBounds(250,250,300,160);
    // Реакція на натискання системної піктограми:
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    // Вікно постійних розмірів:
    setResizable(false);
    // Відключення менеджера компоновки:
    setLayout(null);
    // Створення мітки:
    L=new JLabel();
    // Позиція і розміри мітки:
    L.setBounds(10,10,275,30);
    // Виділення мітки за допомогою ефекту "втискування":
    L.setBorder(
        BorderFactory.createBevelBorder(
            BevelBorder.LOWERED)
    );
    // Реєстрація в мітці обробника для
    // події класу MouseEvent:
    L.addMouseListener(this);
    // Створення текстового поля:
    T=new JTextField();
    // Позиція і розміри поля:
    T.setBounds(10,50,275,30);
}
```



Код програми:

```
// Реєстрація в полі обробника для
// події класу KeyEvent:
T.addKeyListener(this);
// Створення кнопки:
B=new JButton(exit);
// Позиція і розміри кнопки:
B.setBounds(60,90,175,30);
// Відміна режиму відображення рамки фокусу
// для кнопки:
B.setFocusPainted(false);
// Шрифт для тексту кнопки:
B.setFont(new Font(name,Font.PLAIN,size));
// Синій колір для тексту кнопки:
B.setForeground(Color.BLUE);
// Реєстрація в кнопці обробника для
// події класу(ActionEvent:
B.addActionListener(this);
// Реєстрація в кнопці обробника для
// події класу MouseEvent:
B.addMouseListener(this);
// Додавання мітки у вікно:
add(L);
// Додавання поля у вікно:
add(T);
// Додавання кнопки у вікно:
add(B);
// Відображення вікна:
setVisible(true);
}
}
// Головний клас:
class HandlingTextFieldDemo{
    // Головний метод:
    public static void main(String[] args){
        // Створення вікна:
        new MyFrame();
    }
}
```

Приклад: обробник для текстового поля



```
// Імпорт класів:
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
// Клас для створення вікна:
class MyFrame extends JFrame{
    // Мітка:
    private JLabel L;
    // Текстове поле:
    private JTextField T;
    // Кнопка:
    private JButton B;
    // Тип шрифту для кнопки:
    private String name="Arial";
    // Розмір шрифту для кнопки:
    private int size=15;
    // Назва кнопки:
    private String exit="Закрити";
    // Конструктор:
    MyFrame () {
        // Виклик конструктора суперкласу:
        super("Вікно з текстовим полем");
        // Позиція і розміри вікна:
        setBounds (250,250,300,160);
        // Реакція на натискання системної піктограми:
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        // Вікно постійних розмірів:
        setResizable(false);
        // Відключення менеджера компоновки:
        setLayout(null);
        // Створення мітки:
        L=new JLabel();
        // Позиція і розміри мітки:
        L.setBounds(10,10,275,30);
        // Виділення мітки за допомогою ефекту "втискування":
        L.setBorder(
            BorderFactory.createBevelBorder(
                BevelBorder.LOWERED)
        );
    }
}
```

Приклад: використання класів-адаптерів

Розв'язується та сама задача, що і в попередньому випадку, але тепер клас **MyFrame** не реалізує інтерфейси, а замість цього для кожного з компонентів (мітка, поле і кнопка) створюється окремий обробник (для кнопки навіть два). Обробники створюються на основі анонімних класів, які успадковують класи-адаптери. Винятком є обробник події класу **ActionEvent**, який реєструється в кнопці й створюється реалізацією інтерфейсу **ActionListener** (оскільки для цього інтерфейсу немає класу-адаптеру)



Приклад: використання класів-адаптерів

Код програми:

```
// Реєстрація в мітці обробника для
// події класу MouseEvent:
L.addMouseListener(new MouseAdapter() {
    // Метод викликається, коли курсор опиняється
    // над областю мітки:
    public void mouseEntered(MouseEvent e) {
        // Для мітки застосовується ефект "підняття":
        L.setBorder(
            BorderFactory.createBevelBorder(
                BevelBorder.RAISED)
        );
        // Застосування вирівнювання за правим краєм
        // для тексту в мітці:
        L.setHorizontalAlignment(JLabel.RIGHT);
    }
    // Метод викликається, коли курсор миші залишає
    // область мітки:
    public void mouseExited(MouseEvent e) {
        // Застосування до мітки ефекту "втискування":
        L.setBorder(
            BorderFactory.createBevelBorder(
                BevelBorder.LOWERED)
        );
        // Застосування вирівнювання за лівим краєм
        // для тексту в мітці:
        L.setHorizontalAlignment(JLabel.LEFT);
    }
});
// Створення текстового поля:
T=new JTextField();
// Позиція і розміри поля:
T.setBounds(10,50,275,30);
```

```
// Реєстрація в полі обробника для
// події класу KeyEvent:
T.addKeyListener(new KeyAdapter() {
    // Метод викликається у випадку
    // відпускання клавіші на клавіатурі:
    public void keyReleased(KeyEvent e) {
        // До мітки застосовується текст
        // з текстового поля:
        L.setText(T.getText());
    }
});
// Створення кнопки:
B=new JButton(exit);
// Позиція і розміри кнопки:
B.setBounds(60,90,175,30);
// Відміна режиму відображення рамки
// фокусу для кнопки:
B.setFocusPainted(false);
// Шрифт для тексту кнопки:
B.setFont(new Font(name,Font.PLAIN,size));
// Синій колір для тексту кнопки:
B.setForeground(Color.BLUE);
// Реєстрація в кнопці обробника для
// події класу ActionEvent:
B.addActionListener(new ActionListener() {
    // Метод для обробки натискання кнопки:
    public void actionPerformed(ActionEvent e) {
        // Завершення виконання програми:
        System.exit(0);
    }
});
```

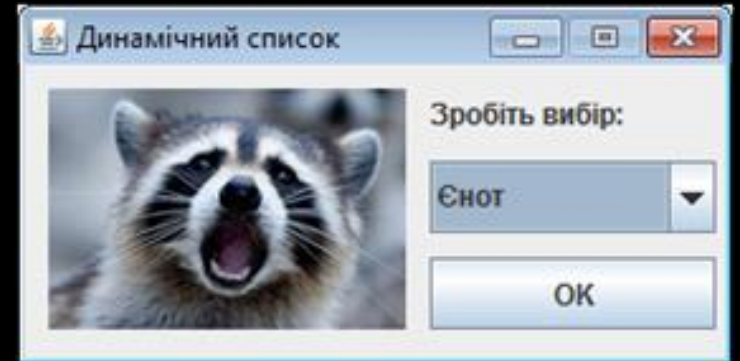
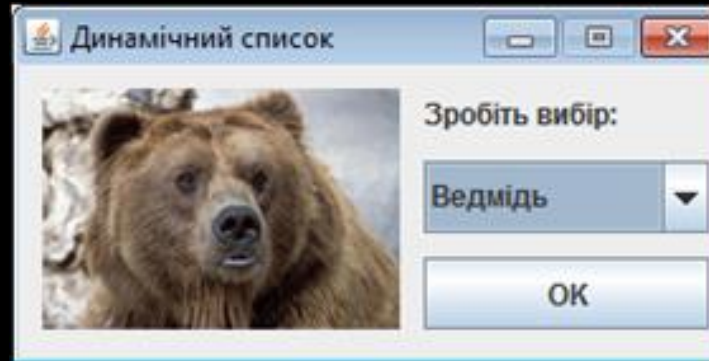
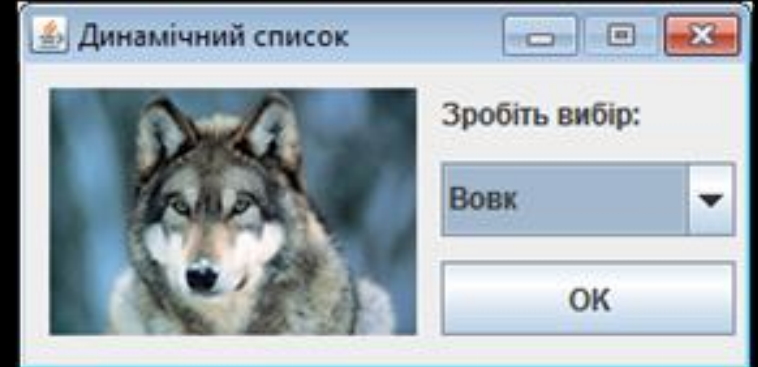
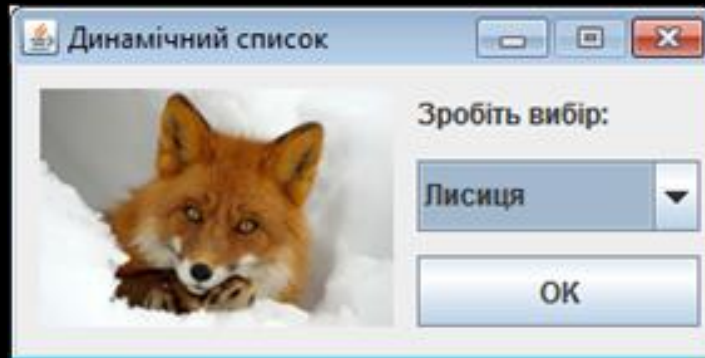
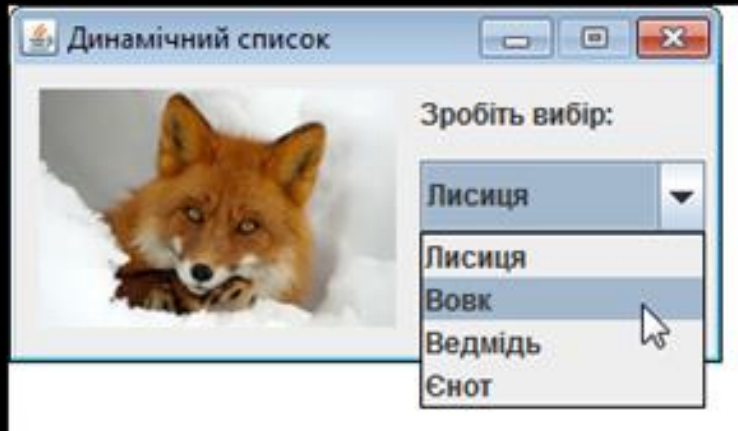


Код програми:

```
// Реєстрація в кнопці обробника для
// події класу MouseEvent:
B.addMouseListener(new MouseAdapter() {
    // Метод викликається, коли курсор миші залишає
    // область кнопки:
    public void mouseExited(MouseEvent e) {
        // Текст (звичайний) для кнопки:
        B.setText(exit);
        // Синій колір для тексту кнопки:
        B.setForeground(Color.BLUE);
        // Шрифт (звичайний) для кнопки:
        B.setFont(new Font(name, Font.PLAIN, size));
    }
    // Метод викликається, коли курсор опиняється
    // над областю кнопки:
    public void mouseEntered(MouseEvent e) {
        // Текст (підкреслений) для кнопки:
        B.setText("<html><u>" + exit + "</u></html>");
        // Червоний колір для тексту кнопки:
        B.setForeground(Color.RED);
        // Шрифт (жирний) для тексту кнопки:
        B.setFont(new Font(name, Font.BOLD, size + 2));
    }
});
// Додавання мітки у вікно:
add(L);
// Додавання поля у вікно:
add(T);
// Додавання кнопки у вікно:
add(B);
// Відображення вікна:
setVisible(true);
}
}
// Головний клас:
class UsingAdaptersDemo {
    // Головний метод:
    public static void main(String[] args) {
        // Створення вікна:
        new MyFrame();
    }
}
```

Приклад: використання класів-адаптерів

Приклад: динамічний список



Приклад: динамічний список

```
// Імпорт класів:
import javax.swing.*;
import java.awt.event.*;
// Клас для створення об'єкта вікна реалізує
// інтерфейс ItemListener:
class MyFrame extends JFrame implements ItemListener{
    // Динамічний список:
    private JComboBox CB;
    // Кнопка:
    private JButton B;
    // Мітка:
    private JLabel L;
    // Масив з назвами тварин:
    private String[] animals=new String[]{
        "Лисиця", "Вовк", "Ведмідь", "Єнот"
    };
    // Масив з назвами файлів з зображеннями:
    private String[] files=new String[]{
        "fox.jpg", "wolf.jpg", "bear.jpg", "raccoon.jpg"
    };
    // Шлях до каталогу з зображеннями:
    private String path="d:/books/pictures/";
    // Масив з зображеннями:
    private ImageIcon[] imgs;
    // Метод викликається у випадку зміни стану списку:
    public void itemStateChanged(ItemEvent e){
        // Мітці присвоюється нове зображення:
        L.setIcon(imgs[CB.getSelectedIndex()]);
    }
    // Конструктор:
    MyFrame(){
        // Налаштування параметрів вікна:
        super("Динамічний список");
        setBounds(250,250,300,150);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setResizable(false);
        setLayout(null);
        // Створення масиву з посилань на об'єкти зображень:
        imgs=new ImageIcon[files.length];
        // Створення об'єктів зображень
        // і заповнення масиву:
        for(int k=0;k<imgs.length;k++){
            imgs[k]=new ImageIcon(path+files[k]);
        }
    }
}
```

```
// Створення мітки з зображенням:
L=new JLabel(imgs[0]);
// Положення і розміри мітки:
L.setBounds(10,10,150,100);
// Додавання мітки у вікно:
add(L);
// Створення і додавання мітки з текстом у вікно:
JLabel lbl=new JLabel("Зробіть вибір:");
lbl.setBounds(170,10,120,20);
add(lbl);
// Створення динамічного списку:
CB=new JComboBox(animals);
// Позиція і розміри списку:
CB.setBounds(170,40,120,30);
// В списку вибирається початковий пункт:
CB.setSelectedIndex(0);
// Реєстрація обробника в списку:
CB.addItemListener(this);
// Додавання списку у вікно:
add(CB);
// Створення кнопки:
B=new JButton("OK");
// Позиція і розміри кнопки:
B.setBounds(170,80,120,30);
// Реєстрація обробника в кнопці:
B.addActionListener(e->System.exit(0));
// Додавання кнопки у вікно:
add(B);
// Відображення вікна на екрані:
setVisible(true);
}
}
// Головний клас:
class UsingComboBoxDemo{
    // Головний метод:
    public static void main(String[] args){
        // Створення об'єкта вікна:
        new MyFrame();
    }
}
```

Дякую за увагу!

Запитання?